

实验四：处理机调度

陈天乐 无 25

2025 年 6 月 3 日

目录

1	实验题目：银行家算法	2
1.1	问题描述	2
1.2	实现要求	2
1.3	程序设计思路	2
1.3.1	算法流程图	2
1.3.2	测试用例结构体定义	3
1.3.3	银行家类定义	3
1.3.4	读取测试用例函数	6
1.3.5	通用测试执行函数	8
1.3.6	主函数	8
1.4	实验测例	8
2	实验结果	12
2.1	算法鲁棒性及效率分析	19
3	思考题	19
4	实验总结	20
5	源代码	21

1 实验题目：银行家算法

本实验选择了银行家算法，操作系统平台选为 Windows，实现语言为 C++。

1.1 问题描述

银行家算法是避免死锁的一种重要方法，将操作系统视为银行家，操作系统管理的资源视为银行家管理的资金，进程向操作系统请求分配资源即企业向银行申请贷款的过程。

请根据银行家算法的思想，编写程序模拟实现动态资源分配，并能够有效避免死锁的发生。

1.2 实现要求

1. 对实现的算法通过流程图进行说明；
2. 设计不少于三组测试样例，需包括资源分配成功和失败的情况；
3. 能够展示系统资源占用和分配情况的变化及安全性检测的过程；
4. 结合操作系统课程中银行家算法理论对实验结果进行分析，验证结果的正确性；
5. 分析算法的鲁棒性及算法效率。

1.3 程序设计思路

1.3.1 算法流程图

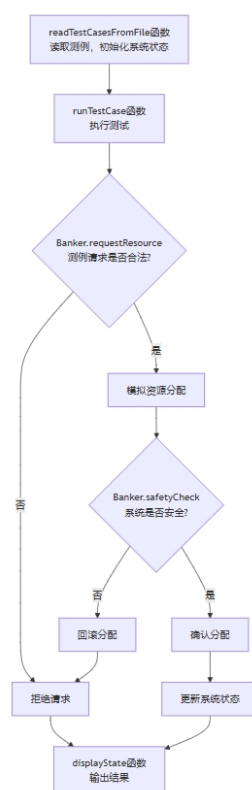


图 1: 银行家算法流程图

银行家算法的程序设计核心是模拟操作系统对资源的动态分配过程，因此需要首先用可用资源向量、最大需求矩阵、分配矩阵和需求矩阵表示系统资源状态。同时实现一个从文件中读取测例的函数用于初始化每个系统的状态。定义一个银行家 Banker 类，在运行测试测例时，银行家对系统发出的资源申请做判断处理，首先是检测这个申请是否合法，然后在申请合法的基础上模拟分配过程。同时需要一个安全性检测函数，在模拟分配过程后中寻找是否存在安全序列，如果存在安全序列，则该申请后系统安全，允许申请；若找不到安全序列，说明该申请后系统进入不安全状态，驳回申请并输出报错。

1.3.2 测试用例结构体定义

```

1 struct TestCase {
2     int processNum;
3     int resourceNum;
4     vector<int> Available;
5     vector<vector<int>> Matrix_MaxNeed;
6     vector<vector<int>> Matrix_Allocation;
7     int pid;
8     vector<int> request;
9 };

```

这一部分代码定义了测例结构体，方便读入文件中的测例。

1.3.3 银行家类定义

```

1 class Banker {
2 private:
3     int processNum;    // 进程数量
4     int resourceNum;   // 资源种类数量
5     vector<int> Available;    // 可用资源向量
6     vector<vector<int>> Matrix_MaxNeed;    // 最大需求矩阵
7     vector<vector<int>> Matrix_Allocation; // 已分配矩阵
8     vector<vector<int>> Matrix_Need;    // 需求矩阵
9
10    vector<int> safetyCheck()    // 安全性检查函数，模拟实现动态资源分配
11    {
12        vector<int> safeSequence_final;
13
14
15        for (int ii = 0; ii < processNum; ii++)
16        {
17            vector<int> Work = Available;    // 工作向量，模拟分配时的可用资源
18            vector<bool> Finish(processNum, false);    // 记录模拟运行的进程是否完成，初始全都未完成
19            vector<int> safeSequence;
20
21            bool flag = true;
22            for (int j = 0; j < resourceNum; j++)
23            {
24                if (Work[j] < Matrix_Need[ii][j])    // 超出了限制
25                {
26                    flag = false;
27                    break;
28                }
29            }
30            if (flag)
31            {
32                for (int j = 0; j < resourceNum; j++)

```

```

33         Work[j] += Matrix_Allocation[i][j];
34         safeSequence.push_back(ii);
35         Finish[ii] = true;
36     }
37     else
38         continue;
39
40     bool found = true;    ///  

41     for (int i = 0; i < processNum; i++)
42     {
43         if (!Finish[i])
44         {
45             bool canallocate = true;
46             for (int j = 0; j < resourceNum; j++)
47             {
48                 if (Work[j] < Matrix_Need[i][j])           ///  

49                 {                                           //超出了限制
50                     canallocate = false;
51                     break;
52                 }
53             }
54
55             if (canallocate)
56             {
57                 for (int j = 0; j < resourceNum; j++)
58                     Work[j] += Matrix_Allocation[i][j];
59                 safeSequence.push_back(i);
60                 Finish[i] = true;
61                 i = -1;
62             }
63
64             }
65
66         }
67     for (bool f : Finish) {
68         if (!f) {
69             ///  

70             found = false;
71         }
72     if (!found)
73         safeSequence_final = {};
74     else
75     {
76         safeSequence_final = safeSequence;
77         break;
78     }
79 }
80 }
81
82 return safeSequence_final;
83 }
84
85 public:
86 Banker(int p, int r, vector<int> av,
87     vector<vector<int>> m, vector<vector<int>> a)
88 : processNum(p), resourceNum(r), Available(av), Matrix_MaxNeed(m), Matrix_Allocation(a)
89 ) {
90     ///  

91     // 计算Need矩阵: Need = MaxNeed - Allocation
92     Matrix_Need.resize(processNum, vector<int>(resourceNum));
93     for (int i = 0; i < processNum; i++) {
94         for (int j = 0; j < resourceNum; j++) {

```

```

94         Matrix_Need[i][j] = Matrix_MaxNeed[i][j] - Matrix_Allocation[i][j];
95         if (Matrix_Need[i][j] < 0) {
96             cerr << "错误：最大需求小于已分配资源！进程" << i << " 资源" << j << " !"
97         }
98         exit(1);
99     }
100 }
101 }
102
103 // 处理资源请求函数，检测合法性
104 bool requestResource(int pid, vector<int> request, vector<string>& log) {
105     for (int j = 0; j < resourceNum; j++) {
106         if (request[j] < 0) {
107             log.push_back("错误：请求不能为负数！");
108             return false;
109         }
110         if (request[j] > Available[j]) {
111             log.push_back("错误：资源不足，需等待资源释放！");
112             return false;
113         }
114         if (request[j] > Matrix_Need[pid][j]) {
115             log.push_back("错误：请求超过需求！");
116             return false;
117         }
118     }
119 }
120
121 //保存现场
122 vector<int> Available_copy = Available;
123 vector<vector<int>> Matrix_Allocation_copy = Matrix_Allocation;
124 vector<vector<int>> Matrix_Need_copy = Matrix_Need;
125
126 // 模拟资源分配
127 for (int j = 0; j < resourceNum; j++) {
128     Available[j] -= request[j];
129     Matrix_Allocation[pid][j] += request[j];
130     Matrix_Need[pid][j] -= request[j];
131 }
132
133 // 输出临时状态
134 log.push_back("\n模拟分配后的临时状态：");
135 log.push_back("可用资源向量： " + vec2string(Available));
136 log.push_back("已分配矩阵:\n " + matrix2string(Matrix_Allocation));
137 log.push_back("需求矩阵:\n " + matrix2string(Matrix_Need));
138
139 // 安全性检查
140 vector<int> safeSequence = safetyCheck();
141 if (!safeSequence.empty()) { //证明有
安全序列
142     log.push_back("\n系统处于安全状态！安全序列为： " + seq2string(safeSequence));
143     return true;
144 }
145 else {
146     log.push_back("\n无安全序列存在，系统不安全，回滚分配...");
147     // 恢复现场
148     Available = Available_copy;
149     Matrix_Allocation = Matrix_Allocation_copy;
150     Matrix_Need = Matrix_Need_copy;
151     return false;
152 }
153 }

```

```

154
155     string seq2string(vector<int> seq) {
156         string res;
157         for (size_t i = 0; i < seq.size(); i++) {
158             res += "P" + to_string(seq[i]);
159             if (i != seq.size() - 1) res += " -> ";
160         }
161         return res;
162     }
163
164     string vec2string(vector<int> vec)
165     {
166         string s = "[";
167         for (size_t i = 0; i < vec.size(); i++) {
168             s += to_string(vec[i]);
169             if (i != vec.size() - 1) s += ", ";
170         }
171         s += "]";
172         return s;
173     }
174
175     string matrix2string(vector<vector<int>>matrix)
176     {
177         string s;
178         for (size_t i = 0; i < matrix.size(); i++) {
179             s += "P" + to_string(i) + ": " + vec2string(matrix[i]) + "\n";
180         }
181         return s;
182     }
183
184     void displayState()
185     {
186         cout << "\n=====当前系统状态=====: \n";
187         cout << "可用资源向量: " << vec2string(Available) << endl;
188         cout << "最大需求矩阵:\n" << matrix2string(Matrix_MaxNeed);
189         cout << "已分配矩阵:\n" << matrix2string(Matrix_Allocation);
190         cout << "需求矩阵:\n" << matrix2string(Matrix_Need);
191     }
192
193 };

```

这一部分是实现银行家算法的核心。其中 Available、Matrix_MaxNeed、Matrix_Aloocation、Matrix_Need 是当前所有可用资源、各进程最大需求矩阵、各进程已分配资源矩阵、各进程仍需资源矩阵。而 safetyCheck 则是整个算法的核心函数。其大概工作机制是对每一个进程进行遍历循环，模拟进程运行结束后的资源分配过程。如果在这个模拟进程中能够找到一个安全序列，那么直接停止循环直接返回安全序列。requestResource 函数就是处理资源请求函数，在每一次资源申请时会调用 safetyCheck 函数检测系统安全性，并会输出模拟资源分配后的情况。倘若遍历了所有可能性也无法找到一个使得所有进程都完成的安全序列，那么这个资源申请就会使得整个系统进入不安全状态，因此驳回该请求并回滚资源状态，同时输出报错信息。seq2string、vec2string、matrix2string 三个函数是便于输出系统状态信息的转换函数。displayState 函数就是用于可视化当前系统状态的函数，输出可用资源向量等信息。

1.3.4 读取测试用例函数

```

1 // 从文件读取测试用例的函数
2 vector<TestCase> readTestCasesFromFile(const string& filename, vector<string>& log) {
3     vector<TestCase> testCases;
4     ifstream file(filename);

```

```
5  if (!file.is_open()) {
6      cerr << "错误：无法打开测试用例文件 " << filename << endl;
7      exit(1);
8  }
9
10 string line;
11 while (getline(file, line)) {
12     if (line.empty() || line.substr(0, 4) == "Test") continue; // 跳过空行和测试用例标题
13
14     TestCase tc;
15     // 读取进程数量
16     if (line.substr(0, 11) == "ProcessNum:") {
17         istringstream iss(line.substr(11));
18         iss >> tc.processNum;
19     }
20     // 读取资源种类数
21     getline(file, line);
22     if (line.substr(0, 12) == "ResourceNum:") {
23         istringstream iss(line.substr(12));
24         iss >> tc.resourceNum;
25     }
26     // 读取可用资源向量
27     getline(file, line);
28     if (line.substr(0, 10) == "Available:") {
29         istringstream iss(line.substr(10));
30         int val;
31         while (iss >> val) {
32             tc.Available.push_back(val);
33         }
34     }
35     // 读取最大需求矩阵
36     getline(file, line);
37     if (line == "Max:") {
38         for (int i = 0; i < tc.processNum; i++) {
39             getline(file, line);
40             istringstream iss(line);
41             vector<int> row;
42             int val;
43             while (iss >> val) {
44                 row.push_back(val);
45             }
46             tc.Matrix_MaxNeed.push_back(row);
47         }
48     }
49     // 读取已分配矩阵
50     getline(file, line);
51     if (line == "Allocation:") {
52         for (int i = 0; i < tc.processNum; i++) {
53             getline(file, line);
54             istringstream iss(line);
55             vector<int> row;
56             int val;
57             while (iss >> val) {
58                 row.push_back(val);
59             }
60             tc.Matrix_Allocation.push_back(row);
61         }
62     }
63     // 读取请求信息
64     getline(file, line);
65     if (line.substr(0, 8) == "Request:") {
66         istringstream iss(line.substr(8));
```

```

67         iss >> tc.pid; // 第一个数是进程ID
68         int val;
69         while (iss >> val) {
70             tc.request.push_back(val);
71         }
72     }
73
74     testCases.push_back(tc);
75     getline(file, line); // 读取测试用例分隔空行
76 }
77 return testCases;
78 }

```

这一部分定义的 readTestcasesFromFile 函数是用于从文件中读取特定格式的测例，生成相对应的当前所有可用资源、各进程最大需求矩阵、各进程已分配资源矩阵、各进程仍需资源矩阵。

1.3.5 通用测试执行函数

```

1 // 通用测试执行函数
2 void runTestCase(const TestCase& tc, vector<string>& log) {
3     cout << "\n===== 执行测试用例 =====\n";
4     Banker banker(tc.processNum, tc.resourceNum, tc.Available, tc.Matrix_MaxNeed, tc.
5         Matrix_Allocation);
6     banker.displayState();
7
8     bool result = banker.requestResource(tc.pid, tc.request, log);
9
10    cout << "\n操作日志: \n";
11    for (string s : log) cout << s << endl;
12    cout << "最终分配结果: " << (result ? "成功" : "失败") << endl;
13    if (result) banker.displayState();
14    log.clear();
15 }

```

这一部分定义的 runTestCase 函数是用于执行测试用例的函数，会输出操作日志中的结果以及最终分配情况。

1.3.6 主函数

```

1 int main() {
2
3     string test_data_file = "C:/Users/skyhappy/Desktop/test4.txt";
4     vector<string> log;
5
6     vector<TestCase> testCases = readTestCasesFromFile(test_data_file, log);
7     for (const auto& tc : testCases) {
8         runTestCase(tc, log);
9     }
10
11    return 0;
12 }

```

主函数中定义测例文件，并运行测例执行函数。

1.4 实验测例

测试文件由若干测例组成。在该实验中，设计的测例为：

```
1 TestCase1: 安全分配成功
2 ProcessNum: 5
3 ResourceNum: 3
4 Available: 3 3 2
5 Max:
6 7 5 3
7 3 2 2
8 9 0 2
9 2 2 2
10 4 3 3
11 Allocation:
12 0 1 0
13 2 0 0
14 3 0 2
15 2 1 1
16 0 0 2
17 Request: 1 1 0 2
18
19 TestCase2: 请求超过最大需求 (失败)
20 ProcessNum: 5
21 ResourceNum: 3
22 Available: 3 3 2
23 Max:
24 7 5 3
25 3 2 2
26 9 0 2
27 2 2 2
28 4 3 3
29 Allocation:
30 0 1 0
31 2 0 0
32 3 0 2
33 2 1 1
34 0 0 2
35 Request: 1 2 1 3
36
37 TestCase3: 分配导致不安全状态 (失败)
38 ProcessNum: 3
39 ResourceNum: 2
40 Available: 1 1
41 Max:
42 3 2
43 2 2
44 2 1
45 Allocation:
46 1 1
47 1 0
48 0 0
49 Request: 2 1 1
50
51 TestCase4: 请求超过可用资源 (失败)
52 ProcessNum: 4
53 ResourceNum: 2
54 Available: 2 1
55 Max:
56 4 3
57 3 2
58 2 2
59 1 1
60 Allocation:
61 2 1
```

```

62 1 0
63 0 1
64 0 0
65 Request: 0 3 1
66
67 TestCase5: 多进程连续请求 (安全)
68 ProcessNum: 3
69 ResourceNum: 3
70 Available: 2 2 2
71 Max:
72 5 5 5
73 4 4 4
74 3 3 3
75 Allocation:
76 1 1 1
77 1 0 1
78 0 1 0
79 Request: 1 1 1 1
80
81 TestCase6: 单进程单资源边界测试 (安全)
82 ProcessNum: 1
83 ResourceNum: 1
84 Available: 5
85 Max:
86 8
87 Allocation:
88 3
89 Request: 0 2
90
91 TestCase7: 进程完成后释放资源 (安全)
92 ProcessNum: 2
93 ResourceNum: 2
94 Available: 1 1
95 Max:
96 3 3
97 2 2
98 Allocation:
99 2 2
100 0 0
101 Request: 0 1 1
102
103 TestCase8: 8进程5资源安全分配 (核心业务场景)
104 ProcessNum: 8
105 ResourceNum: 5
106 Available: 5 3 7 2 4
107 Max:
108 10 8 12 5 7
109 9 7 11 6 8
110 8 9 10 4 5
111 12 10 15 7 9
112 7 5 9 3 6
113 6 8 10 5 7
114 11 9 13 8 10
115 5 4 6 2 3
116 Allocation:
117 3 2 4 1 2
118 2 1 3 2 3
119 4 3 5 1 2
120 5 4 6 3 4
121 2 1 3 1 2
122 1 2 4 2 3
123 3 2 5 3 4

```

```

124 1 1 2 1 1
125 Request: 3 2 1 3 0 2
126
127 TestCase9: 10进程6资源不安全分配（高并发竞争）
128 ProcessNum: 10
129 ResourceNum: 6
130 Available: 2 1 4 1 3 2
131 Max:
132 5 4 7 3 6 5
133 4 3 6 2 5 4
134 6 5 8 4 7 6
135 5 4 7 3 6 5
136 7 6 9 5 8 7
137 3 2 5 1 4 3
138 8 7 10 6 9 8
139 4 3 6 2 5 4
140 5 4 7 3 6 5
141 2 1 3 1 2 1
142 Allocation:
143 3 2 5 2 4 3
144 2 1 4 1 3 2
145 4 3 6 3 5 4
146 3 2 5 2 4 3
147 5 4 7 4 6 5
148 1 1 3 0 2 1
149 6 5 8 5 7 6
150 2 1 4 1 3 2
151 3 2 5 2 4 3
152 1 0 2 0 1 0
153 Request: 5 1 1 2 1 1 1
154
155 TestCase10: 7进程4资源超量请求（边界压力测试）
156 ProcessNum: 7
157 ResourceNum: 4
158 Available: 4 3 5 2
159 Max:
160 9 7 10 6
161 8 6 9 5
162 10 8 12 7
163 7 5 8 4
164 11 9 13 8
165 6 4 7 3
166 5 3 6 2
167 Allocation:
168 5 4 6 3
169 4 3 5 2
170 6 5 8 4
171 3 2 4 1
172 7 6 9 5
173 2 1 3 1
174 1 1 2 1
175 Request: 0 5 4 5 3

```

Listing 1: test.txt

一共设计了 10 个测例，其中会有成功的测例，以及由于找不到安全序列、请求大于需求、请求大于可用资源等失败测例，同时进程的个数和资源个数也会有多样性的体现。

2 实验结果

实验结果已经保存在 result.txt 文件中，结果如下：

```
1 ===== 执行测试用例 =====
2
3 =====当前系统状态=====:
4 可用资源向量: [3, 3, 2]
5 最大需求矩阵:
6 P0: [7, 5, 3]
7 P1: [3, 2, 2]
8 P2: [9, 0, 2]
9 P3: [2, 2, 2]
10 P4: [4, 3, 3]
11 已分配矩阵:
12 P0: [0, 1, 0]
13 P1: [2, 0, 0]
14 P2: [3, 0, 2]
15 P3: [2, 1, 1]
16 P4: [0, 0, 2]
17 需求矩阵:
18 P0: [7, 4, 3]
19 P1: [1, 2, 2]
20 P2: [6, 0, 0]
21 P3: [0, 1, 1]
22 P4: [4, 3, 1]
23
24 操作日志:
25
26 模拟分配后的临时状态:
27 可用资源向量: [2, 3, 0]
28 已分配矩阵:
29 P0: [0, 1, 0]
30 P1: [3, 0, 2]
31 P2: [3, 0, 2]
32 P3: [2, 1, 1]
33 P4: [0, 0, 2]
34
35 需求矩阵:
36 P0: [7, 4, 3]
37 P1: [0, 2, 0]
38 P2: [6, 0, 0]
39 P3: [0, 1, 1]
40 P4: [4, 3, 1]
41
42
43 系统处于安全状态! 安全序列为: P1 -> P3 -> P0 -> P2 -> P4
44 最终分配结果: 成功
45
46 =====当前系统状态=====:
47 可用资源向量: [2, 3, 0]
48 最大需求矩阵:
49 P0: [7, 5, 3]
50 P1: [3, 2, 2]
51 P2: [9, 0, 2]
52 P3: [2, 2, 2]
53 P4: [4, 3, 3]
54 已分配矩阵:
55 P0: [0, 1, 0]
56 P1: [3, 0, 2]
57 P2: [3, 0, 2]
58 P3: [2, 1, 1]
```

```

59 P4: [0, 0, 2]
60 需求矩阵:
61 P0: [7, 4, 3]
62 P1: [0, 2, 0]
63 P2: [6, 0, 0]
64 P3: [0, 1, 1]
65 P4: [4, 3, 1]
66
67 ===== 执行测试用例 =====
68
69 =====当前系统状态=====:
70 可用资源向量: [3, 3, 2]
71 最大需求矩阵:
72 P0: [7, 5, 3]
73 P1: [3, 2, 2]
74 P2: [9, 0, 2]
75 P3: [2, 2, 2]
76 P4: [4, 3, 3]
77 已分配矩阵:
78 P0: [0, 1, 0]
79 P1: [2, 0, 0]
80 P2: [3, 0, 2]
81 P3: [2, 1, 1]
82 P4: [0, 0, 2]
83 需求矩阵:
84 P0: [7, 4, 3]
85 P1: [1, 2, 2]
86 P2: [6, 0, 0]
87 P3: [0, 1, 1]
88 P4: [4, 3, 1]
89
90 操作日志:
91 错误: 请求超过需求!
92 最终分配结果: 失败
93
94 ===== 执行测试用例 =====
95
96 =====当前系统状态=====:
97 可用资源向量: [1, 1]
98 最大需求矩阵:
99 P0: [3, 2]
100 P1: [2, 2]
101 P2: [2, 1]
102 已分配矩阵:
103 P0: [1, 1]
104 P1: [1, 0]
105 P2: [0, 0]
106 需求矩阵:
107 P0: [2, 1]
108 P1: [1, 2]
109 P2: [2, 1]
110
111 操作日志:
112
113 模拟分配后的临时状态:
114 可用资源向量: [0, 0]
115 已分配矩阵:
116 P0: [1, 1]
117 P1: [1, 0]
118 P2: [1, 1]
119
120 需求矩阵:

```

```

121 P0: [2, 1]
122 P1: [1, 2]
123 P2: [1, 0]
124
125
126 无安全序列存在, 系统不安全, 回滚分配...
127 最终分配结果: 失败
128
129 ===== 执行测试用例 =====
130
131 =====当前系统状态=====:
132 可用资源向量: [2, 1]
133 最大需求矩阵:
134 P0: [4, 3]
135 P1: [3, 2]
136 P2: [2, 2]
137 P3: [1, 1]
138 已分配矩阵:
139 P0: [2, 1]
140 P1: [1, 0]
141 P2: [0, 1]
142 P3: [0, 0]
143 需求矩阵:
144 P0: [2, 2]
145 P1: [2, 2]
146 P2: [2, 1]
147 P3: [1, 1]
148
149 操作日志:
150 错误: 资源不足, 需等待资源释放!
151 最终分配结果: 失败
152
153 ===== 执行测试用例 =====
154
155 =====当前系统状态=====:
156 可用资源向量: [2, 2, 2]
157 最大需求矩阵:
158 P0: [5, 5, 5]
159 P1: [4, 4, 4]
160 P2: [3, 3, 3]
161 已分配矩阵:
162 P0: [1, 1, 1]
163 P1: [1, 0, 1]
164 P2: [0, 1, 0]
165 需求矩阵:
166 P0: [4, 4, 4]
167 P1: [3, 4, 3]
168 P2: [3, 2, 3]
169
170 操作日志:
171
172 模拟分配后的临时状态:
173 可用资源向量: [1, 1, 1]
174 已分配矩阵:
175 P0: [1, 1, 1]
176 P1: [2, 1, 2]
177 P2: [0, 1, 0]
178
179 需求矩阵:
180 P0: [4, 4, 4]
181 P1: [2, 3, 2]
182 P2: [3, 2, 3]

```

```

183
184
185 无安全序列存在，系统不安全，回滚分配...
186 最终分配结果：失败
187
188 ===== 执行测试用例 =====
189
190 =====当前系统状态===== :
191 可用资源向量：[5]
192 最大需求矩阵：
193 P0：[8]
194 已分配矩阵：
195 P0：[3]
196 需求矩阵：
197 P0：[5]
198
199 操作日志：
200
201 模拟分配后的临时状态：
202 可用资源向量：[3]
203 已分配矩阵：
204 P0：[5]
205
206 需求矩阵：
207 P0：[3]
208
209
210 系统处于安全状态！安全序列为：P0
211 最终分配结果：成功
212
213 =====当前系统状态===== :
214 可用资源向量：[3]
215 最大需求矩阵：
216 P0：[8]
217 已分配矩阵：
218 P0：[5]
219 需求矩阵：
220 P0：[3]
221
222 ===== 执行测试用例 =====
223
224 =====当前系统状态===== :
225 可用资源向量：[1, 1]
226 最大需求矩阵：
227 P0：[3, 3]
228 P1：[2, 2]
229 已分配矩阵：
230 P0：[2, 2]
231 P1：[0, 0]
232 需求矩阵：
233 P0：[1, 1]
234 P1：[2, 2]
235
236 操作日志：
237
238 模拟分配后的临时状态：
239 可用资源向量：[0, 0]
240 已分配矩阵：
241 P0：[3, 3]
242 P1：[0, 0]
243
244 需求矩阵：

```

```

245 P0: [0, 0]
246 P1: [2, 2]
247
248
249 系统处于安全状态! 安全序列为: P0 -> P1
250 最终分配结果: 成功
251
252 =====当前系统状态=====:
253 可用资源向量: [0, 0]
254 最大需求矩阵:
255 P0: [3, 3]
256 P1: [2, 2]
257 已分配矩阵:
258 P0: [3, 3]
259 P1: [0, 0]
260 需求矩阵:
261 P0: [0, 0]
262 P1: [2, 2]
263
264 ===== 执行测试用例 =====
265
266 =====当前系统状态=====:
267 可用资源向量: [5, 3, 7, 2, 4]
268 最大需求矩阵:
269 P0: [10, 8, 12, 5, 7]
270 P1: [9, 7, 11, 6, 8]
271 P2: [8, 9, 10, 4, 5]
272 P3: [12, 10, 15, 7, 9]
273 P4: [7, 5, 9, 3, 6]
274 P5: [6, 8, 10, 5, 7]
275 P6: [11, 9, 13, 8, 10]
276 P7: [5, 4, 6, 2, 3]
277 已分配矩阵:
278 P0: [3, 2, 4, 1, 2]
279 P1: [2, 1, 3, 2, 3]
280 P2: [4, 3, 5, 1, 2]
281 P3: [5, 4, 6, 3, 4]
282 P4: [2, 1, 3, 1, 2]
283 P5: [1, 2, 4, 2, 3]
284 P6: [3, 2, 5, 3, 4]
285 P7: [1, 1, 2, 1, 1]
286 需求矩阵:
287 P0: [7, 6, 8, 4, 5]
288 P1: [7, 6, 8, 4, 5]
289 P2: [4, 6, 5, 3, 3]
290 P3: [7, 6, 9, 4, 5]
291 P4: [5, 4, 6, 2, 4]
292 P5: [5, 6, 6, 3, 4]
293 P6: [8, 7, 8, 5, 6]
294 P7: [4, 3, 4, 1, 2]
295
296 操作日志:
297
298 模拟分配后的临时状态:
299 可用资源向量: [3, 2, 4, 2, 2]
300 已分配矩阵:
301 P0: [3, 2, 4, 1, 2]
302 P1: [2, 1, 3, 2, 3]
303 P2: [4, 3, 5, 1, 2]
304 P3: [7, 5, 9, 3, 6]
305 P4: [2, 1, 3, 1, 2]
306 P5: [1, 2, 4, 2, 3]

```

```

307 P6: [3, 2, 5, 3, 4]
308 P7: [1, 1, 2, 1, 1]
309
310 需求矩阵:
311 P0: [7, 6, 8, 4, 5]
312 P1: [7, 6, 8, 4, 5]
313 P2: [4, 6, 5, 3, 3]
314 P3: [5, 5, 6, 4, 3]
315 P4: [5, 4, 6, 2, 4]
316 P5: [5, 6, 6, 3, 4]
317 P6: [8, 7, 8, 5, 6]
318 P7: [4, 3, 4, 1, 2]
319
320
321 无安全序列存在, 系统不安全, 回滚分配...
322 最终分配结果: 失败
323
324 ===== 执行测试用例 =====
325
326 =====当前系统状态=====:
327 可用资源向量: [2, 1, 4, 1, 3, 2]
328 最大需求矩阵:
329 P0: [5, 4, 7, 3, 6, 5]
330 P1: [4, 3, 6, 2, 5, 4]
331 P2: [6, 5, 8, 4, 7, 6]
332 P3: [5, 4, 7, 3, 6, 5]
333 P4: [7, 6, 9, 5, 8, 7]
334 P5: [3, 2, 5, 1, 4, 3]
335 P6: [8, 7, 10, 6, 9, 8]
336 P7: [4, 3, 6, 2, 5, 4]
337 P8: [5, 4, 7, 3, 6, 5]
338 P9: [2, 1, 3, 1, 2, 1]
339 已分配矩阵:
340 P0: [3, 2, 5, 2, 4, 3]
341 P1: [2, 1, 4, 1, 3, 2]
342 P2: [4, 3, 6, 3, 5, 4]
343 P3: [3, 2, 5, 2, 4, 3]
344 P4: [5, 4, 7, 4, 6, 5]
345 P5: [1, 1, 3, 0, 2, 1]
346 P6: [6, 5, 8, 5, 7, 6]
347 P7: [2, 1, 4, 1, 3, 2]
348 P8: [3, 2, 5, 2, 4, 3]
349 P9: [1, 0, 2, 0, 1, 0]
350 需求矩阵:
351 P0: [2, 2, 2, 1, 2, 2]
352 P1: [2, 2, 2, 1, 2, 2]
353 P2: [2, 2, 2, 1, 2, 2]
354 P3: [2, 2, 2, 1, 2, 2]
355 P4: [2, 2, 2, 1, 2, 2]
356 P5: [2, 1, 2, 1, 2, 2]
357 P6: [2, 2, 2, 1, 2, 2]
358 P7: [2, 2, 2, 1, 2, 2]
359 P8: [2, 2, 2, 1, 2, 2]
360 P9: [1, 1, 1, 1, 1, 1]
361
362 操作日志:
363
364 模拟分配后的临时状态:
365 可用资源向量: [1, 0, 2, 0, 2, 1]
366 已分配矩阵:
367 P0: [3, 2, 5, 2, 4, 3]
368 P1: [2, 1, 4, 1, 3, 2]

```

```

369 P2: [4, 3, 6, 3, 5, 4]
370 P3: [3, 2, 5, 2, 4, 3]
371 P4: [5, 4, 7, 4, 6, 5]
372 P5: [2, 2, 5, 1, 3, 2]
373 P6: [6, 5, 8, 5, 7, 6]
374 P7: [2, 1, 4, 1, 3, 2]
375 P8: [3, 2, 5, 2, 4, 3]
376 P9: [1, 0, 2, 0, 1, 0]
377
378 需求矩阵:
379 P0: [2, 2, 2, 1, 2, 2]
380 P1: [2, 2, 2, 1, 2, 2]
381 P2: [2, 2, 2, 1, 2, 2]
382 P3: [2, 2, 2, 1, 2, 2]
383 P4: [2, 2, 2, 1, 2, 2]
384 P5: [1, 0, 0, 0, 1, 1]
385 P6: [2, 2, 2, 1, 2, 2]
386 P7: [2, 2, 2, 1, 2, 2]
387 P8: [2, 2, 2, 1, 2, 2]
388 P9: [1, 1, 1, 1, 1, 1]
389
390
391 系统处于安全状态! 安全序列为: P5 -> P0 -> P1 -> P2 -> P3 -> P4 -> P6 -> P7 -> P8 -> P9
392 最终分配结果: 成功
393
394 =====当前系统状态=====:
395 可用资源向量: [1, 0, 2, 0, 2, 1]
396 最大需求矩阵:
397 P0: [5, 4, 7, 3, 6, 5]
398 P1: [4, 3, 6, 2, 5, 4]
399 P2: [6, 5, 8, 4, 7, 6]
400 P3: [5, 4, 7, 3, 6, 5]
401 P4: [7, 6, 9, 5, 8, 7]
402 P5: [3, 2, 5, 1, 4, 3]
403 P6: [8, 7, 10, 6, 9, 8]
404 P7: [4, 3, 6, 2, 5, 4]
405 P8: [5, 4, 7, 3, 6, 5]
406 P9: [2, 1, 3, 1, 2, 1]
407 已分配矩阵:
408 P0: [3, 2, 5, 2, 4, 3]
409 P1: [2, 1, 4, 1, 3, 2]
410 P2: [4, 3, 6, 3, 5, 4]
411 P3: [3, 2, 5, 2, 4, 3]
412 P4: [5, 4, 7, 4, 6, 5]
413 P5: [2, 2, 5, 1, 3, 2]
414 P6: [6, 5, 8, 5, 7, 6]
415 P7: [2, 1, 4, 1, 3, 2]
416 P8: [3, 2, 5, 2, 4, 3]
417 P9: [1, 0, 2, 0, 1, 0]
418 需求矩阵:
419 P0: [2, 2, 2, 1, 2, 2]
420 P1: [2, 2, 2, 1, 2, 2]
421 P2: [2, 2, 2, 1, 2, 2]
422 P3: [2, 2, 2, 1, 2, 2]
423 P4: [2, 2, 2, 1, 2, 2]
424 P5: [1, 0, 0, 0, 1, 1]
425 P6: [2, 2, 2, 1, 2, 2]
426 P7: [2, 2, 2, 1, 2, 2]
427 P8: [2, 2, 2, 1, 2, 2]
428 P9: [1, 1, 1, 1, 1, 1]
429
430 ===== 执行测试用例 =====

```

```
431
432 =====当前系统状态=====;
433 可用资源向量: [4, 3, 5, 2]
434 最大需求矩阵:
435 P0: [9, 7, 10, 6]
436 P1: [8, 6, 9, 5]
437 P2: [10, 8, 12, 7]
438 P3: [7, 5, 8, 4]
439 P4: [11, 9, 13, 8]
440 P5: [6, 4, 7, 3]
441 P6: [5, 3, 6, 2]
442 已分配矩阵:
443 P0: [5, 4, 6, 3]
444 P1: [4, 3, 5, 2]
445 P2: [6, 5, 8, 4]
446 P3: [3, 2, 4, 1]
447 P4: [7, 6, 9, 5]
448 P5: [2, 1, 3, 1]
449 P6: [1, 1, 2, 1]
450 需求矩阵:
451 P0: [4, 3, 4, 3]
452 P1: [4, 3, 4, 3]
453 P2: [4, 3, 4, 3]
454 P3: [4, 3, 4, 3]
455 P4: [4, 3, 4, 3]
456 P5: [4, 3, 4, 2]
457 P6: [4, 2, 4, 1]
458
459 操作日志:
460 错误: 资源不足, 需等待资源释放!
461 最终分配结果: 失败
```

可以自行结合操作系统课程中银行家算法理论对实验结果进行验证, 该程序结果是正确的。

2.1 算法鲁棒性及效率分析

该算法在如资源请求、矩阵初始化等关键逻辑中实现了基本的输入验证, 提升了鲁棒性。并且当模拟资源分配前保存现场, 在分配后发现系统不安全时, 程序会回滚到分配前的状态, 避免系统进入不安全状态, 这是鲁棒性的重要保障。当然, 还有一些潜在的鲁棒性问题, 比如文件读取鲁棒性不足, readTestCasesFromFile 函数假设输入文件严格符合格式, 未设立应对处理文件格式错误的机制, 可能导致解析错误。同时部分错误场景未将错误信息记录到日志中, 不利于问题追溯。

该算法的核心是对系统做安全性检查, 因此时间复杂度主要由 safetyCheck 函数决定。假设进程数为 n , 资源数为 m 。安全检查需要遍历所有进程, 每次遍历需检查每个进程的资源需求是否满足 $O(m)$, 内层循环遍历进程并反复重置索引 $O(n^2)$ 。因此, 安全检查的时间复杂度为 $O(mn^2)$ 。程序的空间复杂度主要由存储系统状态的矩阵 (Available、Matrix_MaxNeed、Matrix_Allocation、Matrix_Need) 决定, 均为 $O(nm)$, 属于合理范围。

3 思考题

1. 银行家算法在实现过程中需注意资源分配的哪些事项才能避免死锁?

从实现过程可以得知, 银行家算法通过模拟分配资源并检查系统是否处于安全状态来避免死锁, 实际上是通过破坏“循环等待条件”来避免死锁的。

在资源分配中首先注意资源申请的合法性，同时在模拟分配后需要检查是否有安全序列存在，如果不存在的话还需要将资源回滚并驳回该资源申请。因此每个进程必须预先声明其最大资源需求量，这个声明值不可以动态更改。而资源分配矩阵必须在安全性检测和模拟分配过程中实时性更新。

4 实验总结

在本次实验中，通过在课上对银行家算法的理论学习，使用 C++ 将其具体实现出来后，对其细节和思想有了更为深入的了解，深入理解死锁避免的核心机制，掌握安全状态检查的具体流程，验证动态资源分配策略如何通过预防循环等待条件避免死锁。此外，在实现程序功能性后，我还自行设计了 10 组测例，用于测试在多种情况下算法的正确性，结果和预期相符合。这也是对我程序设计能力的一种提升，即完成功能性实现后提升程序的鲁棒性，软件工程性意识得到了增强。同时，还画出了算法的流程图，以及对算法做了鲁棒性和效率分析，提高了系统性思维。通过本次实验，我们不仅掌握了一个具体算法，更领悟了计算机系统中“安全性 - 高效性 - 可预测性”的平衡艺术。

最后，感谢老师和助教的批阅！

5 源代码

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <fstream>
5  #include <sstream>
6
7  using namespace std;
8
9  // 测试用例结构体
10 struct TestCase {
11     int processNum;
12     int resourceNum;
13     vector<int> Available;
14     vector<vector<int>> Matrix_MaxNeed;
15     vector<vector<int>> Matrix_Allocation;
16     int pid;
17     vector<int> request;
18 };
19
20 class Banker {
21 private:
22     int processNum;    // 进程数量
23     int resourceNum;   // 资源种类数量
24     vector<int> Available;    // 可用资源向量
25     vector<vector<int>> Matrix_MaxNeed;    // 最大需求矩阵
26     vector<vector<int>> Matrix_Allocation; // 已分配矩阵
27     vector<vector<int>> Matrix_Need;    // 需求矩阵
28
29     vector<int> safetyCheck()    // 安全性检查函数，模拟实现动态资源分配
30     {
31         vector<int> safeSequence_final;
32
33
34         for (int ii = 0; ii < processNum; ii++)
35         {
36             vector<int> Work = Available;    // 工作向量，模拟分配时的可用资源
37
38             vector<bool> Finish(processNum, false);    // 记录模拟运行的进程是否完成，初始全都未完成
39             vector<int> safeSequence;
40
41             bool flag = true;
42             for (int j = 0; j < resourceNum; j++)
43             {
44                 if (Work[j] < Matrix_Need[ii][j])    // 超出了限制
45                 {
46                     flag = false;
47                     break;
48                 }
49             }
50             if (flag)
51             {
52                 for (int j = 0; j < resourceNum; j++)
53                     Work[j] += Matrix_Allocation[ii][j];
54                 safeSequence.push_back(ii);
55                 Finish[ii] = true;
56             }
57             else
58                 continue;

```

```

58
59     bool found = true;    ///  

60     for (int i = 0; i < processNum; i++)  

61     {  

62         if (!Finish[i])  

63         {  

64             bool canallocate = true;  

65             for (int j = 0; j < resourceNum; j++)  

66             {  

67                 if (Work[j] < Matrix_Need[i][j])    ///  

68                 {  

69                     canallocate = false;  

70                     break;  

71                 }  

72             }  

73  

74             if (canallocate)  

75             {  

76                 for (int j = 0; j < resourceNum; j++)  

77                     Work[j] += Matrix_Allocation[i][j];  

78                 safeSequence.push_back(i);  

79                 Finish[i] = true;  

80                 i = -1;  

81             }  

82  

83         }  

84  

85     }  

86     for (bool f : Finish) {  

87         if (!f) {  

88             ///  

89             found = false;  

90         }  

91         if (!found)  

92             safeSequence_final = {};  

93         else  

94         {  

95             safeSequence_final = safeSequence;  

96             break;  

97         }  

98     }  

99 }  

100  

101     return safeSequence_final;  

102 }  

103  

104 public:  

105     Banker(int p, int r, vector<int> av,  

106            vector<vector<int>> m, vector<vector<int>> a)  

107            : processNum(p), resourceNum(r), Available(av), Matrix_MaxNeed(m), Matrix_Allocation(a  

108            ) {  

109  

110         ///  

111         Matrix_Need.resize(processNum, vector<int>(resourceNum));  

112         for (int i = 0; i < processNum; i++) {  

113             for (int j = 0; j < resourceNum; j++) {  

114                 Matrix_Need[i][j] = Matrix_MaxNeed[i][j] - Matrix_Allocation[i][j];  

115                 if (Matrix_Need[i][j] < 0) {  

116                     cerr << "错误: 最大需求小于已分配资源! 进程" << i << " 资源" << j << " !"  

117  

118                     << endl;  

119                     exit(1);  

120                 }  


```

```

118     }
119 }
120 }
121
122 // 处理资源请求函数，检测合法性
123 bool requestResource(int pid, vector<int> request, vector<string>& log) {
124     for (int j = 0; j < resourceNum; j++) {
125         if (request[j] < 0) {
126             log.push_back("错误：请求不能为负数！");
127             return false;
128         }
129         if (request[j] > Available[j]) {
130             log.push_back("错误：资源不足，需等待资源释放！");
131             return false;
132         }
133         if (request[j] > Matrix_Need[pid][j]) {
134             log.push_back("错误：请求超过需求！");
135             return false;
136         }
137     }
138 }
139
140 //保存现场
141 vector<int> Available_copy = Available;
142 vector<vector<int>> Matrix_Allocation_copy = Matrix_Allocation;
143 vector<vector<int>> Matrix_Need_copy = Matrix_Need;
144
145 // 模拟资源分配
146 for (int j = 0; j < resourceNum; j++) {
147     Available[j] -= request[j];
148     Matrix_Allocation[pid][j] += request[j];
149     Matrix_Need[pid][j] -= request[j];
150 }
151
152 // 输出临时状态
153 log.push_back("\n模拟分配后的临时状态：");
154 log.push_back("可用资源向量： " + vec2string(Available));
155 log.push_back("已分配矩阵:\n " + matrix2string(Matrix_Allocation));
156 log.push_back("需求矩阵:\n " + matrix2string(Matrix_Need));
157
158 // 安全性检查
159 vector<int> safeSequence = safetyCheck();
160 if (!safeSequence.empty()) { //证明有
安全序列
161     log.push_back("\n系统处于安全状态！安全序列为： " + seq2string(safeSequence));
162     return true;
163 }
164 else {
165     log.push_back("\n无安全序列存在，系统不安全，回滚分配...");
166     // 恢复现场
167     Available = Available_copy;
168     Matrix_Allocation = Matrix_Allocation_copy;
169     Matrix_Need = Matrix_Need_copy;
170     return false;
171 }
172 }
173
174 string seq2string(vector<int> seq) {
175     string res;
176     for (size_t i = 0; i < seq.size(); i++) {
177         res += "P" + to_string(seq[i]);
178         if (i != seq.size() - 1) res += " -> ";

```

```

179     }
180     return res;
181 }
182
183 string vec2string(vector<int> vec)
184 {
185     string s = "[";
186     for (size_t i = 0; i < vec.size(); i++) {
187         s += to_string(vec[i]);
188         if (i != vec.size() - 1) s += ", ";
189     }
190     s += "]";
191     return s;
192 }
193
194 string matrix2string(vector<vector<int>>matrix)
195 {
196     string s;
197     for (size_t i = 0; i < matrix.size(); i++) {
198         s += "P" + to_string(i) + ": " + vec2string(matrix[i]) + "\n";
199     }
200     return s;
201 }
202
203 void displayState()
204 {
205     cout << "\n=====当前系统状态===== \n";
206     cout << "可用资源向量: " << vec2string(Available) << endl;
207     cout << "最大需求矩阵:\n" << matrix2string(Matrix_MaxNeed);
208     cout << "已分配矩阵:\n" << matrix2string(Matrix_Allocation);
209     cout << "需求矩阵:\n" << matrix2string(Matrix_Need);
210 }
211
212 };
213
214 // 从文件读取测试用例的函数
215 vector<TestCase> readTestCasesFromFile(const string& filename, vector<string>& log) {
216     vector<TestCase> testCases;
217     ifstream file(filename);
218     if (!file.is_open()) {
219         cerr << "错误: 无法打开测试用例文件 " << filename << endl;
220         exit(1);
221     }
222
223     string line;
224     while (getline(file, line)) {
225         if (line.empty() || line.substr(0, 4) == "Test") continue; // 跳过空行和测试用例标题
226
227         TestCase tc;
228         // 读取进程数量
229         if (line.substr(0, 11) == "ProcessNum:") {
230             istringstream iss(line.substr(11));
231             iss >> tc.processNum;
232         }
233         // 读取资源种类数
234         getline(file, line);
235         if (line.substr(0, 12) == "ResourceNum:") {
236             istringstream iss(line.substr(12));
237             iss >> tc.resourceNum;
238         }
239         // 读取可用资源向量
240         getline(file, line);

```

```

241     if (line.substr(0, 10) == "Available:") {
242         istream iss(line.substr(10));
243         int val;
244         while (iss >> val) {
245             tc.Available.push_back(val);
246         }
247     }
248     // 读取最大需求矩阵
249     getline(file, line);
250     if (line == "Max:") {
251         for (int i = 0; i < tc.processNum; i++) {
252             getline(file, line);
253             istream iss(line);
254             vector<int> row;
255             int val;
256             while (iss >> val) {
257                 row.push_back(val);
258             }
259             tc.Matrix_MaxNeed.push_back(row);
260         }
261     }
262     // 读取已分配矩阵
263     getline(file, line);
264     if (line == "Allocation:") {
265         for (int i = 0; i < tc.processNum; i++) {
266             getline(file, line);
267             istream iss(line);
268             vector<int> row;
269             int val;
270             while (iss >> val) {
271                 row.push_back(val);
272             }
273             tc.Matrix_Allocation.push_back(row);
274         }
275     }
276     // 读取请求信息
277     getline(file, line);
278     if (line.substr(0, 8) == "Request:") {
279         istream iss(line.substr(8));
280         iss >> tc.pid; // 第一个数是进程ID
281         int val;
282         while (iss >> val) {
283             tc.request.push_back(val);
284         }
285     }
286
287     testCases.push_back(tc);
288     getline(file, line); // 读取测试用例分隔空行
289 }
290 return testCases;
291 }
292
293 // 通用测试执行函数
294 void runTestCase(const TestCase& tc, vector<string>& log) {
295     cout << "\n===== 执行测试用例 =====\n";
296     Banker banker(tc.processNum, tc.resourceNum, tc.Available, tc.Matrix_MaxNeed, tc.
297         Matrix_Allocation);
298     banker.displayState();
299
300     bool result = banker.requestResource(tc.pid, tc.request, log);
301
302     cout << "\n操作日志: \n";

```

```
302     for (string s : log) cout << s << endl;
303     cout << "最终分配结果: " << (result ? "成功" : "失败") << endl;
304     if (result) banker.displayState();
305     log.clear();
306 }
307
308 int main() {
309
310     string test_data_file = "C:/Users/skyhappy/Desktop/test4.txt";
311     vector<string> log;
312
313     vector<TestCase> testCases = readTestCasesFromFile(test_data_file, log);
314     for (const auto& tc : testCases) {
315         runTestCase(tc, log);
316     }
317
318     return 0;
319 }
```

Listing 2: main.cpp